

In the format provided by the authors and unedited.

Reinforcement learning with analogue memristor arrays

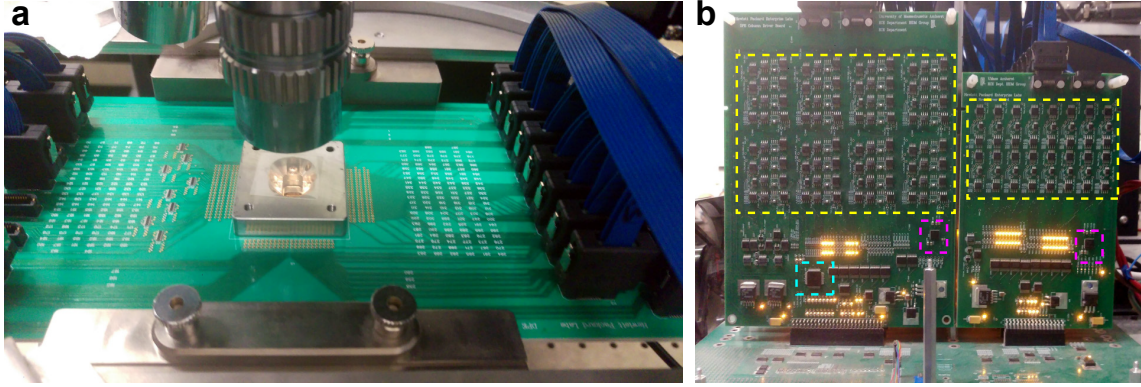
Zhongrui Wang^{1,8}, Can Li^{1,8}, Wenhao Song¹, Mingyi Rao¹, Daniel Belkin¹, Yunning Li¹, Peng Yan¹, Hao Jiang¹, Peng Lin¹, Miao Hu², John Paul Strachan³, Ning Ge³, Mark Barnell⁴, Qing Wu⁴, Andrew G. Barto⁵, Qinru Qiu⁶, R. Stanley Williams⁷, Qiangfei Xia^{1*} and J. Joshua Yang^{1*}

¹Department of Electrical and Computer Engineering, University of Massachusetts, Amherst, MA, USA. ²Binghamton University, Binghamton, NY, USA.

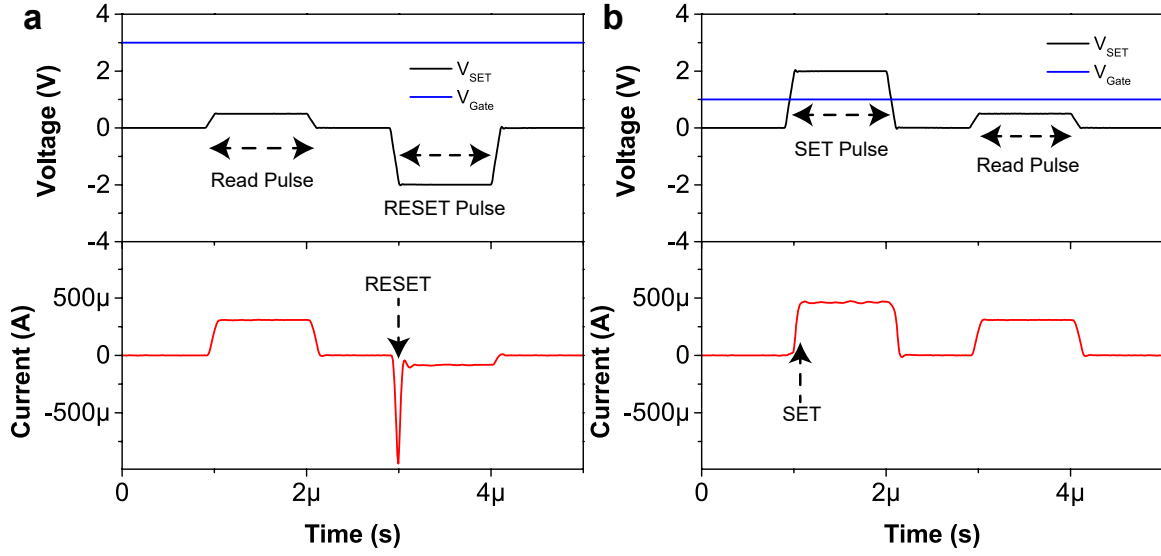
³Hewlett Packard Labs, Hewlett Packard Enterprise, Palo Alto, CA, USA. ⁴Air Force Research Laboratory, Information Directorate, Rome, NY, USA.

⁵College of Information and Computer Sciences, University of Massachusetts, Amherst, MA, USA. ⁶Department of Electrical Engineering and Computer Science, Syracuse University, Syracuse, NY, USA. ⁷Department of Electrical and Computer Engineering, Texas A&M University, College Station, TX, USA.

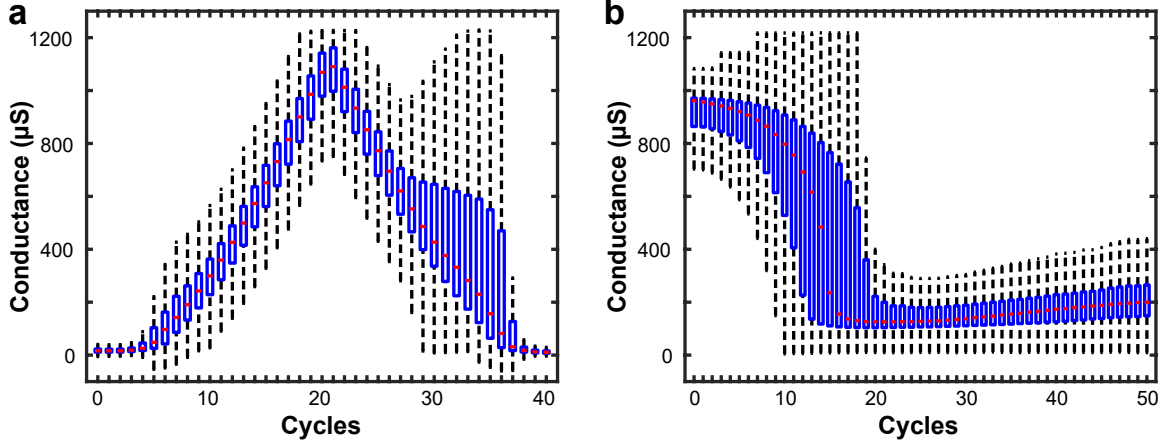
⁸These authors contributed equally: Zhongrui Wang, Can Li. *e-mail: qxia@umass.edu; jjyang@umass.edu



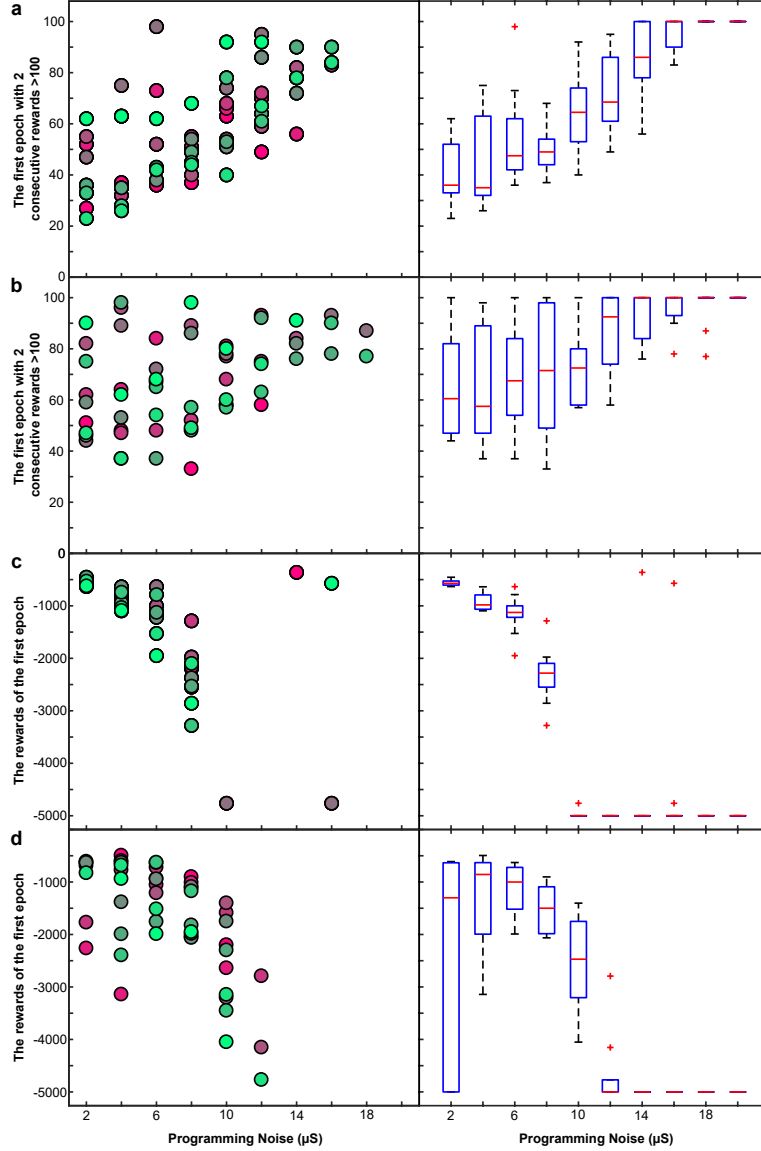
Supplementary Figure 1 | Measurement system for the 1-Transistor 1-Memristor (1T1R) memristor array. **a**, Top view of the 128×64 probe card that is connected to the print circuit boards (PCBs) of the measurement system via high speed co-axial cables (blue). **b**, Front view of the column PCB (left) and row PCB (right). Each column PCB consists of a multi-channel digital-analogue converter (DAC, magenta box) to source voltages to the gates of transistors and top electrodes (TE) of the memristors. The column PCBs are also equipped with arrays of trans-impedance amplifiers (TIAs, yellow box), multiplexers (MUXs, yellow box), and analogue-digital converter (ADCs, cyan box) to sense currents. The row boards consist of arrays of MUXs and latches (yellow box) interfacing with DACs (cyan box) to source voltages to the bottom electrodes (BE) of memristors. All column and row boards are mounted on the same motherboard. The motherboard comes with a microcontroller unit (MCU) interfaced to a digital computer via serial-port communication.



Supplementary Figure 2 | Real-time monitored programming current of a single 1T1R cell. **a**, The monitored current in a depression (RESET) transition. The gate voltage (blue) was constantly 3V. The RESET pulse (black, applied to the TE of the memristor) was -2.5V. The BE of the 1T1R cell is grounded. **b**, The immediately followed potentiation (SET) transition. The gate voltage (blue) was constantly 1V to apply the current compliance. The SET pulse (black) was 2V. The SET transition is of a negligible delay and duration. All data were measured with a Keithley B1530.

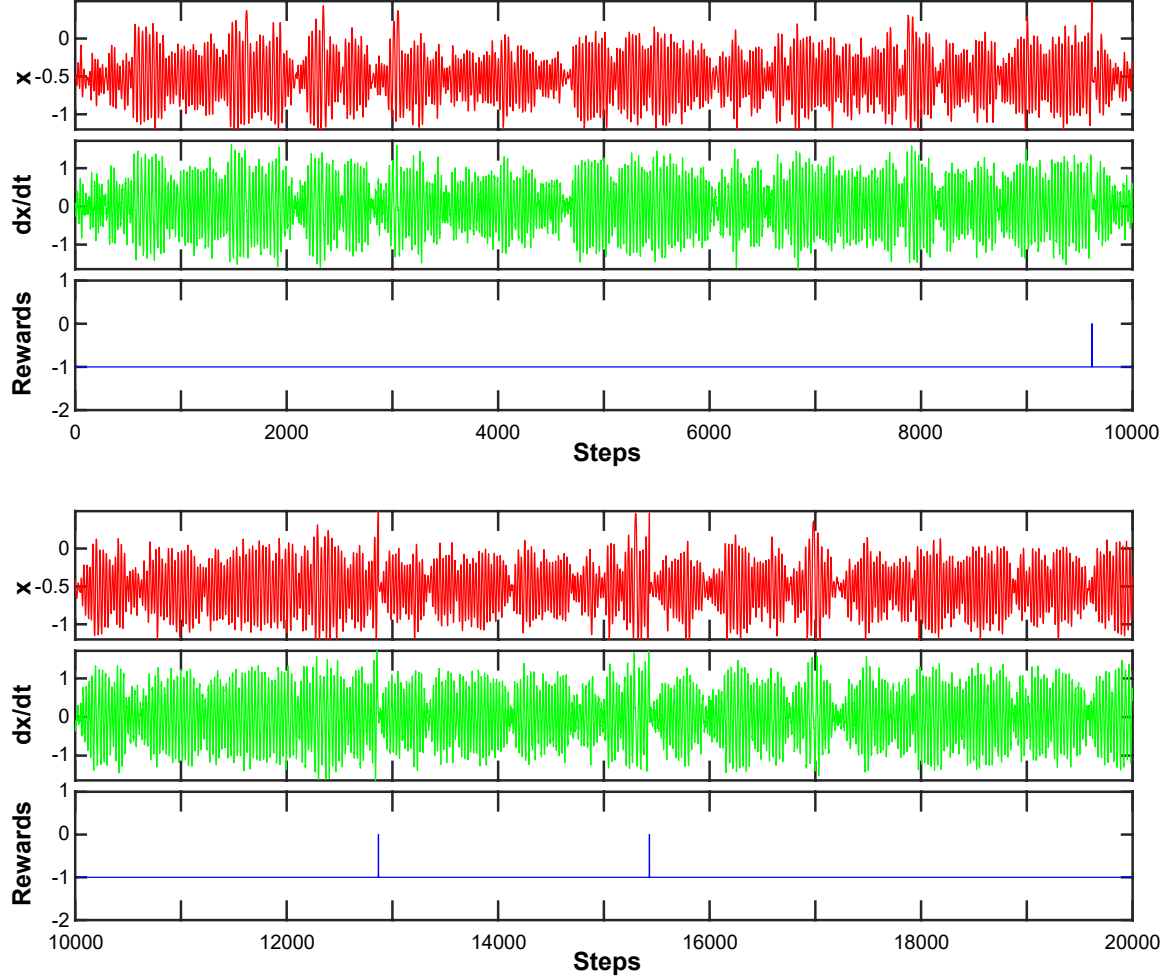


Supplementary Figure 3 | Comparison of analogue programming schemes of the 1T1R memristor array. **a**, Potentiation and depression with linearly varying gate voltages of the series transistors. Statistical distribution of the conductance of all 8,192 memristor synapses after receiving 20 potentiation operations (with the gate voltage ramped linearly from 0.5V to 1.5V in 20 cycles) and 20 depression operations (with the gate voltage ramped linearly from 1.5V to 0.5V in 20 cycles). The tuning is linear and symmetric. This two-pulse write-without-verification programming scheme has speed advantage over the write-and-verify programming scheme at a reasonable accuracy. **b**, Depression with linearly varying memristor RESET (row) voltages V_{RESET} . The amplitude of V_{RESET} ramped linearly from 0.9 V to 2.2V in 50 cycles. All memristors were initialized to their low resistance state (LRS). The tuning shows less linearity and symmetry compared to that in **a**. The red dots indicate the medians, and the bottom and top edges of the blue box indicate the 25th and 75th percentiles, respectively. The dashed lines extend to the most extreme data points.

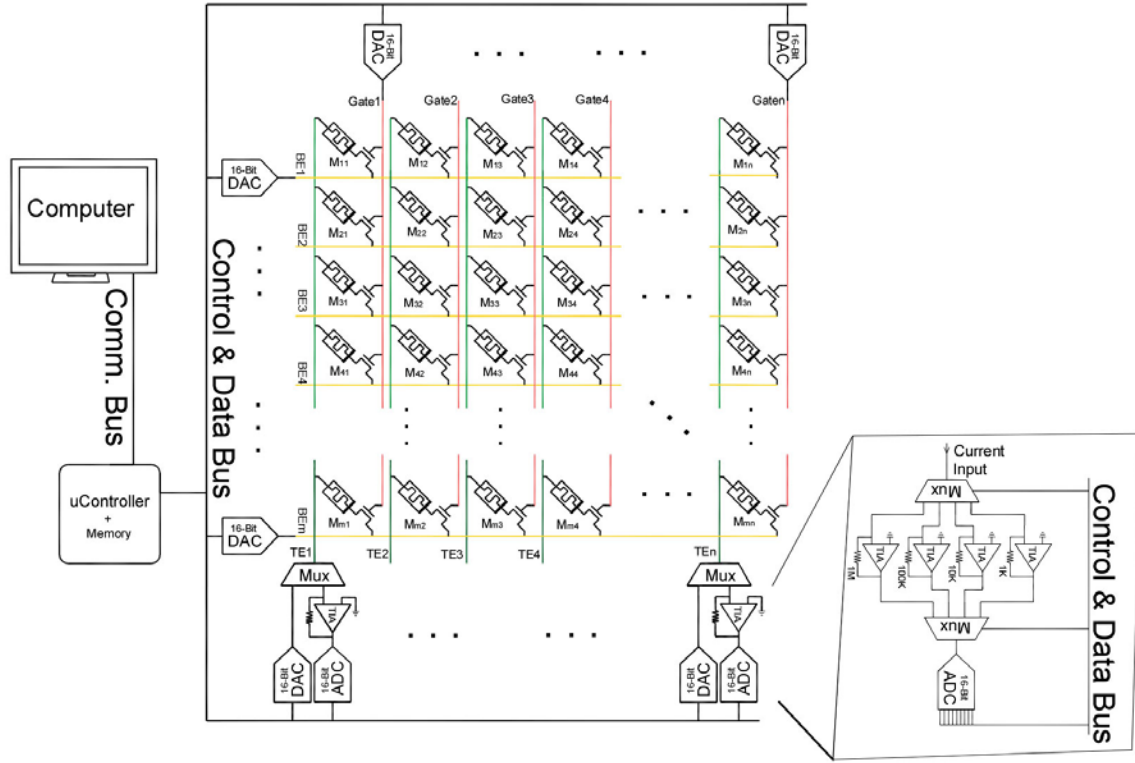


Supplementary Figure 4 | The memristor programming noise impact to the learning performance. **a,b**, The number of episodes the agent takes to achieve two consecutive rewards that are larger than 100 for the cart-pole environment at different programming noise levels, where **a** is with uniform weight initialization and **b** is with normal distributed initial weight. Dots with same colour are of the same initial weights. The dots larger than 100 are not shown. The right panel shows the statistical information of the scattered dots in the left panel, assuming 100 episodes for the dots not shown. The increasing programming noise clearly increases the number of episodes needed to achieve the targeted performance for both weight initialization schemes. **c,d**, The negative rewards that the agent receives in the first episode of the mountain car environment at different

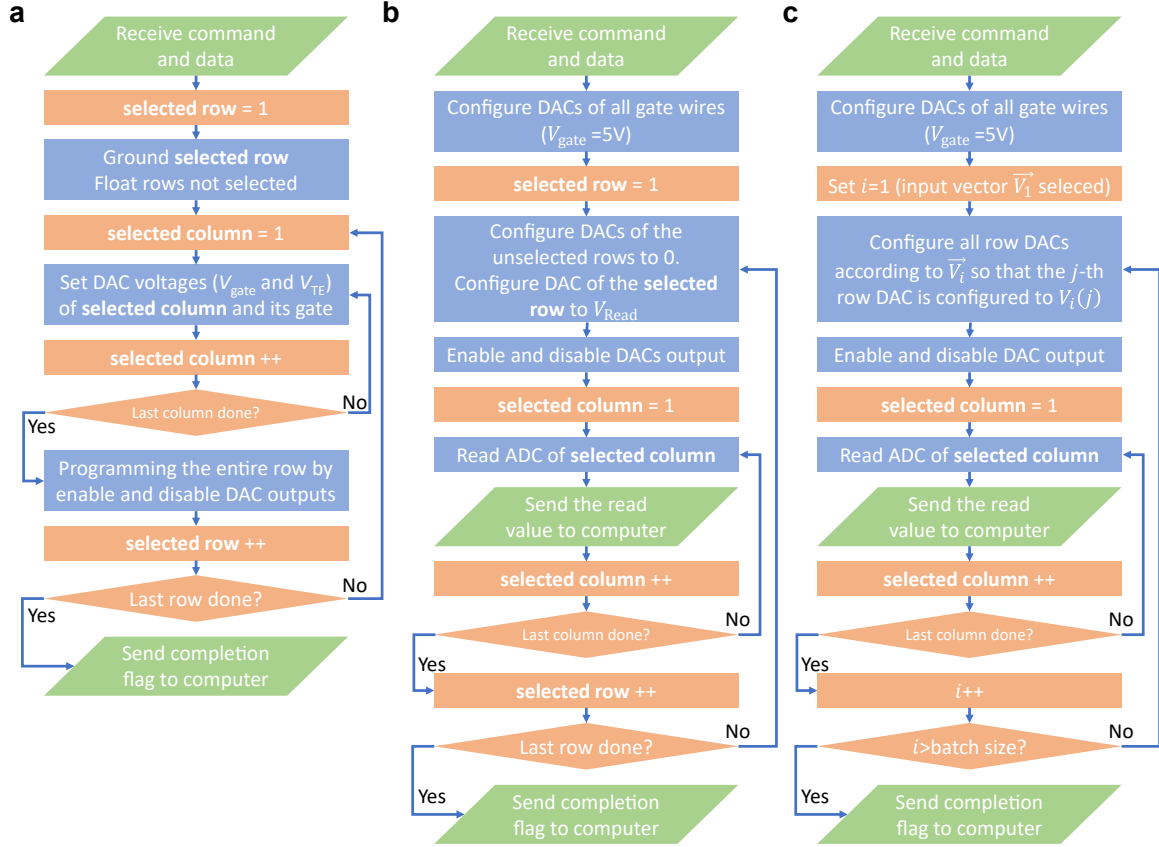
programming noise levels, where **c** is with uniform weight initialization and **d** is with normally distributed initial weight. The dots smaller than -5000 are not shown. The right panel shows the statistical information of the scattered dots in the left panel, assuming -5000 rewards for the dots not shown here. The increasing programming noise clearly increases the negative rewards, making the first episode to be solved with more steps.



Supplementary Figure 5 | Pre-train memory for the mountain car. The pre-train experience consists of 20,000 game steps with random actions. All actions occur with equal probability. The chance that the car could win the game with random walk is very low. There are in total 3 times that the car hits the goal region and receives 0 rewards over the 20,000 steps. The pre-train experience is added to the memory of the learning agent before actual training. Note the agent cannot learn the strategy to push the car out of the valley unless both types of transitions, hitting or not hitting the flag, have been acquired. The mountain car environment setting is likely to produce a transition that hits the flag target with a few thousand random walks. Thus, the pre-training memory, acquired first from random actions, facilitates the solving of the mountain car problem. In addition, the pre-train experience has also been widely used to accelerate the reinforcement learning of a variety of tasks as reported in Ref. ¹.



Supplementary Figure 6 | Functional block diagram of the measurement system performing the roles of hidden neurons. The neurons consist of 3 modules. The post-synaptic neuron modules interface with the TEs or column wires. Each of them mainly consists of 4 TIAs with different gains to cover a wide range of currents. (Note that only one TIA will be active during the operation.) The activation function module is implemented on software of the computer. The pre-synaptic neuron modules interface with the row wires. Each of them mainly consists of a DAC channel.



Supplementary Figure 7 | The potentiation programming, weight read-out, and batch vector-matrix multiplication flowcharts of the 1T1R memristor array. **a**, The potentiation programming flowchart. The 1T1R array is programmed row-by-row. For each row, the MCU sets the DAC output voltages of all the column wires (constant V_{TE}) and gate wires (target conductance dependent V_{gate}). **b**, The weight readout flowchart. The weights of the array are read row-by-row. In each time step, the selected row is biased to V_{Read} while the rest rows are grounded. The MCU reads the ADCs of all columns in each time step and sends the readings back to the computer. All gate wires are set to $V_{gate} = 5V$ in all time steps. **c**, The batch vector-matrix multiplication flowchart. In the i -th time step, the row DACs are configured to produce the i -th input vector $\overline{V_i}$. The MCU reads the ADCs of all columns in each time step and sends the reading back to the computer. All gate wires are set to V_{gate} in all time steps.

<pre> Initialize replay memory Initialize weights and biases Set t to 1 for epoch = 1: maximum epoch number Initialize s_t to a random legal state while r_t is not the epoch ending reward, or the first step of the epoch if t is not 1 Sample a minibatch (s_j, a_j, r_j, s_{j+1}) from replay memory Physical inference on all s_j and s_{j+1} in minibatch together with s_t on 1T1R array Set y_j = $\begin{cases} r_j, & \text{if the sample of the minibatch is the last step of an epoch} \\ r_j + \gamma \cdot \max_{a_{j+1} \in [a]} (Q(s_{j+1}, a_{j+1})), & \text{otherwise} \end{cases}$ Calculate gradients by minimizing $\sum_{j \in \text{minibatch}} (y_j - Q(s_j, a_j))^2$ using RMSprop Physical update of conductance of 1T1R array with the calculated weight gradients end Generate a random number in the range of (0,1) if the random number < ε, or t = 1 Select a random action a_t else Select the action with $\max_{a_t \in [a]} Q(s_t, a_t)$ end Execute action a_t on emulator, receive reward r_t and next state s_{t+1} Store (s_t, a_t, r_t, s_{t+1}) to replay memory Set ε = ε_{min} + (ε_{max} - ε_{min})exp(-λt) Increase t by 1 end end </pre>	<pre> Initialize replay memory Initialize action-value function Q with random weights and biases Initialize target action-value function Q̂ = Q (same weights and biases) for epoch = 1: maximum epoch number Initialize frame sequence {x₁} and preprocessed input vector φ₁ = φ({x₁}) for t = 1: maximum epoch duration if the random number < ε, or t = 1 Select a random action a_t else Select the action with $\max_{a_t \in [a]} Q(\phi_t, a_t)$ end Execute action a_t on emulator, receive reward r_t and frame sequence {x_{t+1}} Set φ_{t+1} = φ({x_{t+1}}) Store (φ_t, a_t, r_t, φ_{t+1}) to replay memory Sample a minibatch (φ_j, a_j, r_j, φ_{j+1}) from replay memory Set y_j = $\begin{cases} r_j, & \text{if the sample of the minibatch is the last step of an epoch} \\ r_j + \gamma \cdot \max_{a_{j+1} \in [a]} (Q(\phi_{j+1}, a_{j+1})), & \text{otherwise} \end{cases}$ Calculate gradients by minimizing $\sum_{j \in \text{minibatch}} (y_j - Q(s_j, a_j))^2$ using RMSprop Set Q̂ = Q every C step Set ε = ε_{min} + (ε_{max} - ε_{min})exp(-λt) end end </pre>
---	---

Supplementary Figure 8 | Comparison of the pseudo-codes of the reinforcement learning algorithms. The code used in this study (with orange background) is modified based on the report by Mnih et al.¹ (green background).

Supplementary Table 1 | Comparison between the reported memristor array with the 1T1R Pd/HfO₂/Ta memristor array used in this work (see the second column).

Institution	UMass ^{2,3} (this work)	HP ⁴	Tsinghua ⁵	ASU ⁶	IBM Almaden ^{7,8}	UCSB ^{9,10}		UMich ¹¹	Southampton ¹²
Memristor	Pd/HfO ₂ /Ta	Pt/HfO ₂ /Ta	TiN/HfAl _y O _x /TaO _x /TiN	TiN/HfO ₂ /TaO _x /TiN	Doped Ge ₂ Se ₂ Te ₅	Pt/Al ₂ O ₃ /TiO ₂ /Ti/Pt		W/WO _x /Pd/Au	Pt/TiO ₂ /Pt
Memristor size	4×4 μm	4×4 μm	N. A.	N. A.	N. A.	200×200 nm		500×500 nm	N. A.
Transistor Node Size	2 μm	2 μm	1.2 μm	130 nm	180 nm	No Transistor		No Transistor	No Transistor
Array size	128×64	128×64	128×8	512×1024	500×661	12×12	20×20	32×32	4×2 (wired)
In-crossbar weighted sum	Yes	Yes	Yes	N. A.	No	Yes		Yes	Yes
Synapse Precision	5-bit/64-level	5-bit/64-level	5% Error with V _{sl} =1.5V; 35% Error with V _{sl} =2.5V	binary weight	maximum variation ~7μS	N. A.	7.4% error	~1μS standard deviation / 9-level	N. A.
Resistance Range	100-900 μS	100-700 μS	10-60 μS	9.1-100 μS	2-20 μS	10-100 μS	12-142 μS	0-5 μS	145-175 μS

Supplementary Table 2 | The transistor operation schemes in different memristor manipulations.

Memristor Manipulation	V_G (Gate Wire)	V_S (Row Wire)	V_D (Transistor / memristor junction)
Memristor potentiation	$[0.6V, 1.5V]$	$0V$	The drain connects to the 2.5V column DAC via the memristor.
Memristor depression	$5V$	$1.7V$	The drain connects to the ground via the memristor.
Weight readout		$0.2V$	
Vector-matrix multiplication		$[-0.2V, 0.2V]$	
Deselection of a memristor	$0V$	N. A.	N. A.

Supplementary Table 3 | List of the hyperparameters used for the cart-pole and mountain-car games.

	Hyperparameters	Value (cart-pole)	Value (mountain car)	Description
Reinforcement learning algorithm	minibatch size	80	80	Number of training samples for each Q-network update is computed.
	replay memory size	20000	20000	Size of the buffer to memorize this number of most recent game steps.
	discount factor	0.99	0.99	gamma used in the Q-network update
	initial exploration	1	1	Initial value of ϵ in ϵ -greedy policy.
	final exploration	0.01	0.01	Final value of ϵ in ϵ -greedy policy.
	exploration decay rate	0.001	0.001	The exponential decay factor of exploration in ϵ -greedy policy.
	replay start size	0	20000	A uniform random policy is run for this number of steps before learning starts. The resulting experience is stored in memory.
Fully connected neural network	learning rate	0.0005	0.0005	The learning rate used by RMSprop.
	momentum	0	0	The gradient momentum used by RMSprop.
	decay	0.9	0.9	Squared gradient (denominator) momentum used by RMSprop.
	eps	10^{-8}	10^{-8}	Denominator shift used by RMSprop.
Hard ware interfa cing	conductance to weight ratio	0.00015	0.00025	The ratio between the change of conductance to the change of weight.
	conductance to gate voltage ratio	0.0001	0.000125	The ratio between the change of conductance to the change of gate voltage.
	minimal potentiation gate voltage	0.6	0.6	Minimal gate voltage used to potentiate a memristor.
	maximum potentiation gate voltage	1.5	1.5	Maximum gate voltage used to potentiate a memristor.
	potentiation voltage	2.5	2.5	Positive voltage applied to the top electrodes to potentiate a memristor.
	depression gate voltage	5	5	Gate voltage used to depress a memristor.
	depression voltage	1.7	1.7	Positive voltage applied to the bottom electrodes to depress a memristor.
	read voltage	0.2	0.2	The read voltage used to sense the conductance of the memristor.

Supplementary Table 4 | The 8-bit operation throughput, thermal design power (TDP), and dynamic random-access memory (DRAM) bandwidth of the Google Tensor Processing Unit (TPU) Gen. 1 and Intel Xeon E5-2699 v3 based systems.¹³

	8-b Throughput	TDP	DRAM bandwidth
Google TPU Gen. 1	$T_{\text{TPU}} = 92 \text{ TOPS}$	$P_{\text{TPU}} = 75 \text{ W}$	$B_{\text{TPU}} = 34 \text{ GB/s}$
Intel Xeon E5-2699 v3	$T_{\text{CPU}} = 2.6 \text{ TOPS}$	$P_{\text{CPU}} = 145 \text{ W}$	$B_{\text{CPU}} = 51 \text{ GB/s}$

Supplementary Table 5 | The architecture of the deep-Q network reported by Mnih et al.¹, where all convolutional layers are expanded to fully connected layers with the same number of input and output neurons.

Layer	Input Neurons	Output Neurons	No. of weights (or multiplications / additions per forward pass)
1	$n_{\text{input_dim}}$ $= 84 \times 84 \times 4$	$20 \times 20 \times 32$	361,267,200
2	$20 \times 20 \times 32$	$9 \times 9 \times 64$	66,355,200
3	$9 \times 9 \times 64$	$7 \times 7 \times 64$	16,257,024
4	$7 \times 7 \times 64$	512	1,605,632
5	512	$n_{\text{output_dim}} = 18$	9,216
Total		$n_{\text{neurons}} = 21,650$	$n_{\text{weights}} = 445,494,272$

Supplementary Table 6 | Energy cost breakdown in a forward pass of a projected hybrid analogue-digital system.

	Energy per operation	No. of instances
Memristor read energy	$E_{\text{mem_read}} = 40 \text{ fJ}$	$n_{\text{weights}} = 445,494,272$
DAC intrinsic energy cost	$E_{\text{DAC}} = 250 \text{ pJ}$	$n_{\text{DAC}} = n_{\text{input_dim}} = 28,224$
TIA intrinsic energy cost	$E_{\text{TIA}} = 2 \text{ pJ}$	$n_{\text{TIA}} = n_{\text{neurons}} = 21,650$
ADC energy cost	$E_{\text{ADC}} = 12.6 \text{ pJ}$	$n_{\text{ADC}} = 2$

Supplementary Table 7 | Breakdown of energy cost, time cost, computational complexity, and time complexity of the hybrid analogue-digital system, the Google TPU Gen. 1 based system, and the Intel Xeon E5-2699 v3 based systems in performing a single episode of the deep-Q reinforcement learning reported by Mnih et al.¹

Steps		Hybrid analogue-digital system	Google TPU Gen. 1 based system	Intel Xeon E5-2699 v3 based system	Computational complexity	Time complexity for hybrid analogue-digital system
Replay	Forward passes	~1.59mJ ~640ns	~46.5mJ ~620μs	~3.18J ~21.9ms	$O(n_{\text{weights}} \times n_{\text{batch}})$	$O(n_{\text{batch}})$
	Bellman equation	~145nJ ~1ns			$O(n_{\text{batch}})$	
	Error backpropagation	~1.59mJ ~640ns	~46.5mJ ~620μs	~3.18J ~21.9ms	$O(n_{\text{weights}} \times n_{\text{batch}})$	$O(n_{\text{batch}})$
	Gradients	~24.8mJ ~171μs			$O(n_{\text{weights}})$	
	Trainable parameter update	~56.8mJ ~141μs	~579mJ ~13.1ms	~579mJ ~8.74ms	$O(n_{\text{weights}})$	$O(\max(n_{\text{input_dim}}^l))$
Action picking	Forward pass	~24.9μJ ~10.0ns	~726μJ ~9.68μs	~49.7mJ ~343μs	$O(n_{\text{weights}})$	$O(1)$
Total		~84.8mJ ~314μs	~698mJ ~14.5ms	~7.01J ~53.1ms	$O(n_{\text{weights}} \times n_{\text{batch}})$ $+ O(n_{\text{weights}})$ $+ O(n_{\text{batch}})$	$O(n_{\text{batch}})$ $+ O(n_{\text{weights}})$ $+ O(\max(n_{\text{input_dim}}^l))$ $+ O(1)$

Supplementary Note 1: Performance analysis of the hybrid analogue-digital system in comparison with the conventional digital systems.

In the following, based on information accessible to us, we present a simple estimation of the performance (energy cost, time cost, computational complexity, and time complexity) of a projected hybrid analogue-digital system, Google Tensor Processing Unit (TPU) Gen. 1 based system, and Intel Xeon E5-2699 v3 based system in implementing a real-world reinforcement learning problem.

For simplicity, we assume the Google TPU Gen. 1 and the Intel Xeon E5-2699 v3 work at their peak throughputs, peak memory bandwidths, and thermal design powers (TDPs).¹³ (see Supplementary Table 4) The dynamic random-access memory (DRAM) writing energy is assumed to be $E_{\text{DRAM}} = 1.3\text{nJ}$ per byte using the reported data of 45nm technology.¹⁴ The off-chip DRAM writing time for Google TPU Gen. 1 and Intel Xeon E5-2699 v3 based systems is ignored assuming the latency is dominated by the memory bandwidth.

The performance will be benchmarked by a single episode of the reinforcement learning with the deep-Q network reported by Mnih et al.¹ To make the comparison straightforward and fair, we assume all convolutional layers have been expanded into fully connected layers with the same number of input and output neurons. (see Supplementary Table 5) The resultant deep-Q network consists of $n_{\text{weights}} = n_{\text{multiplications}} = n_{\text{additions}} = 445,494,272$ trainable weights, or the same number of multiplications / additions per forward pass.

1 Experience Replay: Forward pass with all pre-transition states s_j and post-transition states s_{j+1}

Hybrid analogue-digital system: The Joule heat dissipated by a single memristor (also part of the energy consumption of the DACs or TIAs) is $E_{\text{mem_read}} = \frac{0.2V^2}{10k\Omega} \times 10\text{ns} = 40\text{fJ}$ per forward pass, given the average conductance of memristors $10k\Omega$, the inference voltage $0.2V$, and the voltage pulse width 10ns . (Note that 10ns inference pulse is much longer than the worst-case RC time constant). The transistor gate voltages are constant during inference, which could be generated by an external source rather than DACs.

The hidden neurons could be implemented entirely in the analogue domain¹⁰, while only the input and output neurons are hooked to DACs and ADCs, respectively, to interface with the digital system. Since the low-power DACs¹⁵ (12-bit, 250MHz sampling rate) and TIAs¹⁶ typically consume much less energy than that to drive memristors, the maximum intrinsic energy cost of a single DAC and TIA per forward pass could be estimated as $E_{\text{DAC}} = 25\text{mW} \times 10\text{ns} = 250\text{pJ}$ and $E_{\text{TIA}} = 0.2\text{mW} \times 10\text{ns} = 2\text{pJ}$, respectively. Notice the DAC operating power of Ref. 15 is based on the 180nm node technology and 12-bit resolution, which could be further reduced with transistor scaling and fewer bits. In addition, the 18 output neurons could have their TIA signals converted to digits with $n_{\text{ADC}} = \text{ceil}\left(\frac{n_{\text{output_dim}}}{10\text{ns} \times 1\text{GHz}}\right) = 2$ 6-bit ADCs¹⁷ converting at 1GHz, with energy cost $E_{\text{ADC}} = 1.26\text{mW} \times 10\text{ns} = 12.6\text{pJ}$ per ADC per forward pass.

Therefore, the time cost of the entire minibatch is $t_{\text{mem_1}} = 10\text{ns} \times n_{\text{batch}} \times 2 = 640\text{ns}$. Note that each sample of the minibatch consists of a pre-transition state and a post-transition state, contributing to two forward passes.

The forward pass energy (sum of memristor Joule heating, DAC intrinsic energy cost, TIA intrinsic energy cost, and ADC energy cost, see Supplementary Table 6) is $E_{\text{mem_1}} = (E_{\text{mem_read}} \times n_{\text{weights}} + E_{\text{DAC}} \times n_{\text{DAC}} + E_{\text{TIA}} \times n_{\text{TIA}} + E_{\text{ADC}} \times n_{\text{ADC}}) \times n_{\text{batch}} \times 2 \approx 1.59\text{mJ}$.

Google TPU Gen. 1 based system: The estimated time cost of this step according to the given throughput is $t_{\text{TPU_1}} = (n_{\text{multiplications}} + n_{\text{additions}}) \times n_{\text{batch}} \times 2/T_{\text{TPU}} \approx 620\mu\text{s}$.

The estimated energy cost of this step is $E_{\text{TPU_1}} = P_{\text{TPU}} \times t_{\text{TPU_1}} \approx 46.5\text{mJ}$.

Intel Xeon E5-2699 v3 based system: The estimated time cost of this step according to the given throughput is $t_{\text{CPU_1}} = (n_{\text{multiplications}} + n_{\text{additions}}) \times n_{\text{batch}} \times 2/T_{\text{CPU}} \approx 21.9\text{ms}$.

The estimated energy cost of this step is $E_{\text{CPU_1}} = P_{\text{CPU}} \times t_{\text{CPU_1}} \approx 3.18\text{J}$.

Computational complexity: The computational complexity of the entire minibatch is $O(n_{\text{weights}} \times n_{\text{batch}})$, since $n_{\text{weights}} = n_{\text{multiplications}} = n_{\text{additions}}$.

Time complexity: For the hybrid analogue-digital system, the time complexity is $O(n_{\text{batch}})$ since all MACs of a single forward pass are performed in parallel. For the Google TPU Gen. 1 and Intel Xeon E5-2669 v3 based systems, the time complexity

remains $O(n_{\text{weights}} \times n_{\text{batch}})$ because of the assumed constant throughputs. (Practically, Google TPU Gen. 1 could parallelize the forward pass with the Matrix Multiply Unit. Similarly, Intel Xeon E5-2699 v3 could gain parallelism with the multiple core design and the multiple pipelines within each execution engine.)

2 Experience Replay: Bellman equation

The outputs of the step 1 are used by the Bellman equation for the loss calculation, which is always done in the digital domain with an arithmetic logic unit (ALU). Note that both the hybrid analogue-digital system and the Google TPU Gen. 1 based system rely on the ALUs of the hosting systems. Here the energy and time costs of this step are based on the throughput and the TDP of the Intel Xeon E5-2699 v3.

All systems: The Bellman equation performs one comparison, one multiplication, and one addition for every sample of the minibatch.

The execution time will be limited by the clocking frequency of the Intel Xeon E5-2699 v3, which is assumed to be $t_{\text{mem}_2} = t_{\text{TPU}_2} = t_{\text{CPU}_2} = 3 \times n_{\text{batch}}/T_{\text{CPU}} \approx 1\text{ns}$.

The rough energy cost of this step is $E_{\text{mem}_2} = E_{\text{TPU}_2} = E_{\text{CPU}_2} = P_{\text{CPU}} \times t_{\text{CPU}_2} \approx 145\text{nJ}$.

Computational complexity: The computational complexity of a minibatch is $O(n_{\text{batch}})$.

Time complexity: The time complexity remains $O(n_{\text{batch}})$ because of the assumed constant throughput.

3 Experience Replay: Error backpropagation

The backward pass would bear almost the same energy-speed efficiency with the forward pass in step 2 with the error backpropagation being entirely performed with the memristor arrays using bi-directional hidden neurons. Therefore, $t_{\text{mem}_3} \approx t_{\text{mem}_1}$, $t_{\text{TPU}_3} \approx t_{\text{TPU}_1}$, $t_{\text{CPU}_3} \approx t_{\text{CPU}_1}$, $E_{\text{mem}_3} \approx E_{\text{mem}_1}$, $E_{\text{TPU}_3} \approx E_{\text{TPU}_1}$, and $E_{\text{CPU}_3} \approx E_{\text{CPU}_1}$.

4 Experience Replay: Gradients of trainable parameters

The backpropagated error is used to determine the change of each trainable parameter via elementwise operations, which is always done in the digital domain with an ALU. Notice that both the hybrid analogue-digital system and the Google TPU Gen. 1 based system rely on the ALUs of the hosting systems. (The current programming scheme needs additional access to lastly used programming voltages, which could be avoided by alternative

programming schemes.¹⁸⁾ Here the energy and time costs of this step are based on the throughput and the TDP of the Intel Xeon E5-2699 v3.

All systems: For each trainable parameter, its gradient is computed by a single multiplication. Note extra operations could be incurred by the specific optimizer.

The time to compute the gradients could be estimated as $t_{\text{mem}_4} = t_{\text{TPU}_4} = t_{\text{CPU}_4} = n_{\text{weights}}/T_{\text{CPU}} \approx 171\mu\text{s}$, assuming no bias to be trained.

The rough energy cost of this step is $E_{\text{mem}_4} = E_{\text{TPU}_4} = E_{\text{CPU}_4} = P_{\text{CPU}} \times t_{\text{CPU}_4} \approx 24.8\text{mJ}$.

Computational complexity: The computational complexity per minibatch is thus dependent on the number of trainable parameters $O(n_{\text{weights}})$.

Time complexity: The time complexity remains $O(n_{\text{weights}})$ because of the assumed constant throughput.

5 Experience Replay: Trainable parameter update

All trainable parameters are updated once per minibatch, based on the calculated gradients.

Hybrid analogue-digital system: The memristor used in this study could be programmed by a 2.2V 5ns pulse (limited by the instrument)¹⁹, at an energy cost $E_{\text{mem_write}} = 2.2\text{V}^2/10\text{k}\Omega \times 5\text{ns} = 2.42\text{pJ}$ per programming. Notice that the programming voltage (voltage across 1T1R cells) is constant that could be generated by an external source rather than DACs. The transistor gate voltages during programming shall be produced by DACs. The DAC intrinsic energy cost for weight writing is $E_{\text{DAC_write}} = 25\text{mW} \times 5\text{ns} = 125\text{pJ}$. The total number of working instances of DACs is $n_{\text{DAC_write}} = n_{\text{weights}} = 445,494,272$. Therefore, the energy cost to update all trainable parameters is $E_{\text{mem}_5} = E_{\text{mem_write}} \times n_{\text{weights}} + E_{\text{DAC_write}} \times n_{\text{DAC_write}} \approx 56.8\text{mJ}$. Since the memristor weight update could be partially parallelized in the sense that the memristors of the same row could be updated in a single time step. Also, different layer of the neural network could be updated simultaneously. Therefore, the programming time is determined by the layer with the largest input dimension which is $\max(n_{\text{input_dim}}^l) = 28,244$ (see Supplementary Table 5), where $n_{\text{input_dim}}^l$ denotes the number of input neurons of l -th layer. The total programming

duration is therefore $t_{\text{mem}_5} = \max(n_{\text{input_dim}}^l) \times 5\text{ns} \approx 141\mu\text{s}$ for the hybrid analog-digital system.

Google TPU Gen. 1 based system: Here we assume the Google TPU Gen. 1 could write the DRAM without going through the hosting system.

Based on the memory bandwidth of the TPU (see Supplementary Table 4), the time to update all trainable parameters is $t_{\text{TPU}_5} = n_{\text{weights}}/B_{\text{TPU}} \approx 13.1\text{ms}$.

The off-chip DRAM writing energy is $E_{\text{TPU}_5} = E_{\text{DRAM}} \times n_{\text{weights}} \approx 579\text{mJ}$.

Intel Xeon E5-2699 v3 based system: Based on the memory bandwidth of the Intel Xeon E5-2699 v3 (see Supplementary Table 4), the time to update all trainable parameters is $t_{\text{CPU}_5} = n_{\text{weights}}/B_{\text{CPU}} \approx 8.74\text{ms}$.

The off-chip DRAM writing energy is $E_{\text{CPU}_5} = E_{\text{DRAM}} \times n_{\text{weights}} \approx 579\text{mJ}$.

Computational Complexity: The trainable parameter update complexity is proportional to the number of the trainable parameters $O(n_{\text{weights}})$.

Time Complexity: For Google TPU Gen. 1 and Intel Xeon E5-2699 v3 based systems, the DRAM controller and the memory data bus are sequential with assumed constant memory bandwidths. Therefore, the time complexity retains $O(n_{\text{weights}})$. For the hybrid analog-digital system, the time complexity is reduced to $O(\max(n_{\text{input_dim}}^l))$ because of the partially parallel writing capability.

6 Action Picking: Forward pass with the current state vector s_t

Hybrid analogue-digital system: The single forward pass time is $t_{\text{mem}_6} = 10\text{ns}$.

The single forward pass energy (sum of memristor Joule heating, DAC intrinsic energy cost, TIA intrinsic energy cost, and ADC energy cost, see Supplementary Table 6) is roughly $E_{\text{mem}_6} = E_{\text{mem_read}} \times n_{\text{weights}} + E_{\text{DAC}} \times n_{\text{DAC}} + E_{\text{TIA}} \times n_{\text{TIA}} + E_{\text{ADC}} \times n_{\text{ADC}} \approx 24.9\mu\text{J}$.

Google TPU Gen. 1 based system: The estimated computation time of this step according to the given throughput is $t_{\text{TPU}_6} = (n_{\text{multiplications}} + n_{\text{additions}})/T_{\text{TPU}} \approx 9.68\mu\text{s}$.

The estimated energy cost of this step is thus $E_{\text{TPU}_6} = P_{\text{TPU}} \times t_{\text{TPU}_6} \approx 726\mu\text{J}$.

Intel Xeon E5-2699 v3 based system: The estimated computation time of this step according to the given throughput is $t_{\text{CPU}_6} = (n_{\text{multiplications}} + n_{\text{additions}})/T_{\text{CPU}} \approx 343\mu\text{s}$.

The estimated energy cost of this step is thus $E_{\text{CPU}_6} = P_{\text{CPU}} \times t_{\text{CPU}_6} \approx 49.7\text{mJ}$.

Computational complexity: The computational complexity per minibatch is $O(n_{\text{weights}})$.

Time complexity: For the hybrid analogue-digital system, the time complexity is $O(1)$ since all MACs of the entire network are done in a single step. For the Google TPU Gen. 1 and Intel Xeon E5-2669 v3 based systems, the time complexity remains $O(n_{\text{weights}})$ because of the assumed constant throughputs. (Practically, Google TPU Gen. 1 could parallelize the forward pass with the Matrix Multiply Unit. Similarly, Intel Xeon E5-2699 v3 could gain parallelism with the multiple core design and the multiple pipelines within each execution engine.)

7 Summary of the Comparison

Supplementary Table 7 summarizes the energy cost, time cost, computational complexity, and time complexity breakdown of the hybrid analogue-digital system, the Google TPU Gen. 1 based system, and the Intel Xeon E5-2699 v3 based system in performing a single episode of the reinforcement learning. This simple comparison indicates that the hybrid analogue-digital system could potentially provide $\sim 46\times$ ($\sim 8\times$) and $\sim 169\times$ ($\sim 83\times$) improvements of the time (energy) efficiency over Google TPU Gen. 1 and Intel Xeon E5-2699 v3 based systems, respectively, based on the Pd/HfO₂/Ta memristors demonstrated in this work. (Note that the equivalent bits per memristor is 4-5 in this work, which is less than the 8-bit precision of the transistor-based systems.)

According to Supplementary Table 7, majority of the time and energy saving comes from the forward passes and the trainable parameter updates. This is because the memristors of the hybrid analogue-digital system play dual roles. They not only replace DRAM for storing trainable parameters but also perform vector-matrix multiplications as MACs at where the data are stored, which breaks the von Neumann bottleneck with in-memory computing. On the contrary, the speed of the transistor based neural network accelerators like Google TPU Gen. 1 is limited by the scale of its Matrix Multiply Unit (i.e. 256×256), while the energy efficiency is constrained by the off-chip DRAM accessing.

We also note the time and energy efficiency of the hybrid analogue-digital system could be further boosted by the high-speed low-power ADC and DAC operations and optimized gradients calculations with digital systems. In addition, further improvements in memristor operations parameters, such as the conductance range, operation speed, and programming accuracy can potentially result in another order of magnitude improvement in speed-energy efficiency.

In conclusion, our finding reveals that the real-world reinforcement learning algorithms could be significantly benefited by the hybrid analogue-digital systems in terms of speed-energy efficiency, thanks to the in-memory analogue computing enabled by the 1T1R memristor arrays.

References

- 1 Mnih, V. *et al.* Human-level control through deep reinforcement learning. *Nature* **518**, 529-533, (2015).
- 2 Li, C. *et al.* Efficient and self-adaptive in-situ learning in multilayer memristor neural networks. *Nat. Commun.* **9**, 2385, (2018).
- 3 Li, C. *et al.* Analogue signal and image processing with large memristor crossbars. *Nat. Electron.* **1**, 52-59, (2018).
- 4 Hu, M. *et al.* Memristor-Based Analog Computation and Neural Network Classification with a Dot Product Engine. *Adv. Mater.* **30**, 1705914, (2018).
- 5 Yao, P. *et al.* Face classification using electronic synapses. *Nat. Commun.* **8**, 15199, (2017).
- 6 Yu, S. *et al.* in *Electron Devices Meeting (IEDM), 2016 IEEE International*. 16.12. 11-16.12. 14 (IEEE).
- 7 Burr, G. W. *et al.* Experimental Demonstration and Tolerancing of a Large-Scale Neural Network (165 000 Synapses) Using Phase-Change Memory as the Synaptic Weight Element. *IEEE Trans. Elect. Dev.* **62**, 3498-3507, (2015).
- 8 Le Gallo, M. *et al.* Mixed-precision in-memory computing. *Nat. Electron.* **1**, 246-253, (2018).
- 9 Prezioso, M. *et al.* Training and operation of an integrated neuromorphic network based on metal-oxide memristors. *Nature* **521**, 61-64, (2015).
- 10 Bayat, F. M. *et al.* Implementation of multilayer perceptron network with highly uniform passive memristive crossbar circuits. *Nat. Commun.* **9**, 2331, (2018).
- 11 Sheridan, P. M. *et al.* Sparse coding with memristor networks. *Nat. Nanotechnol.* **12**, 784-789, (2017).
- 12 Serb, A. *et al.* Unsupervised learning in probabilistic neural networks with multi-state metal-oxide memristive synapses. *Nat. Commun.* **7**, 12611, (2016).
- 13 Jouppi, N. P. *et al.* in *Proceedings of the 44th Annual International Symposium on Computer Architecture*. 1-12 (ACM).
- 14 Horowitz, M. in *Solid-State Circuits Conference Digest of Technical Papers (ISSCC), 2014 IEEE International*. 10-14 (IEEE).

- 15 Chi, J.-H., Chu, S.-H. & Tsai, T.-H. in *ESSCIRC, 2010 Proceedings of the*. 222-225 (IEEE).
- 16 Atef, M., Atef, A. & Abbas, M. in *Circuits and Systems (ISCAS), 2016 IEEE International Symposium on*. 2423-2426 (IEEE).
- 17 Choo, K. D., Bell, J. & Flynn, M. P. in *Solid-State Circuits Conference (ISSCC), 2016 IEEE International*. 460-461 (IEEE).
- 18 Chen, P.-Y. *et al.* in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*. 194-199 (IEEE Press).
- 19 Jiang, H. *et al.* Sub-10 nm Ta Channel Responsible for Superior Performance of a HfO₂ Memristor. *Sci. Rep.* **6**, 28525, (2016).